

STAT 310 Final Exam

Problem 1.

The algorithm that I will implement in this problem is **Algorithm 17.4** (Bound-Constrained Lagrangian Method) from the Numerical Optimization book by Nocedal and Wright.

This algorithm requires a solution of a subproblem (17.50) within each iteration, i.e. minimizing the augmented Lagrangian subject to bound constraints. In order to solve this subproblem I will be following a projected gradient approach combined with a line search as outlined in Section 18.6 of the book.

In order to find an approximate solution of the quadratic subproblem, I will be using **Algorithm 16.5** (Gradient Projection for QP) as outlined in section 16.7 of the book. Further, I calculate the Cauchy point as outlined in the same section by projecting a point x onto the feasible region.

The function I use to generate the random nonlinear program is
"RandomNonLinearProgram(m, n, s)"

- Inputs:
 - m, n: the dimensions of the problem
 - s: random seed
- Outputs:
 - [c, Q0, D0, D, a]: the matrices and vectors that define the objective function and the equality constraints

The function that will solve the bound constrained subproblem is:
"BCLMethod(c, Q0, D0, D, a, gamma, maxIter)"

- Inputs:
 - C, Q0, D0, D, a: the matrices and vectors of the objective and constraints
 - gamma: the non convexity parameter
 - maxIter: the maximum number of iterations for the main algorithm
- Outputs:
 - x: the Solution vector
 - lambda: Lagrange multipliers for equality constraints
 - mu: - the Penalty parameter

Note that the bound constraints $-1 < x < 1$ are not passed as parameters, but are explicitly defined in the algorithm since we do not change the constraints.

Within this algorithm I make calls to other algorithms / functions which are below:

- *"solveSubproblem(x_k, lambda, mu, c, Q0, D0, D, a, gamma, omega)"*
 - Solves the subproblem (17.50) for each iteration of the BCLMethod algorithm
- *"gradient_projection_method(x_k, g, H)"*
 - Returns the approximate solution of the quadratic subproblem for use within solveSubproblem, via a gradient projection method
- *"computeCauchyPoint(x_k, g_k, B_k, l, u)"*
 - Computes the Cauchy Point for use within the gradient_projection_method
- *"projected_CG(x_c, l, u, G, c)"*
 - Performs the Projected Conjugate Gradient method (Algorithm 16.2) for use within the gradient_projection_method

Within use of all the algorithms, I have the following functions to compute the function value, gradient, and Hessian of the Augmented Lagrangian at a given x and fixed λ and μ :

- *"augmentedLagrangian(x, lambda, mu, c, Q0, D0, D, a, gamma)"*
- *"gradLagrangian(x, lambda, mu, c, Q0, D0, D, a, gamma)"*
- *"computeHessian(x, lambda, mu, Q0, D0, D, a, gamma)"*

Further I have the following helper function to project a vector onto a given bounds:
"projectOntoBounds(x, l, u)"

Using the algorithm:

We can generate a random nonlinear program, set a value for gamma, and use the BCL_Method function to solve the problem as follows:

```
>>
m = 5;
n = 2 * m;
s = 6; % Random Seed

% Non-convexity parameter
gamma = 0;
% Maximum number of iterations
maxIter = 10;

[c, Q0, D0, D, a] = RandomNonLinearProgram(m, n, s);
x_star = BCLMethod(c, Q0, D0, D, a, gamma, maxIter)
>>
```

The output when running the script is below:

```
>>Iteration: 1
Objective value: 0.0000
Iteration: 2
Objective value: -7.9117
Iteration: 3
Objective value: -11.1886
Iteration: 4
Objective value: -9.6977
Iteration: 5
Objective value: -9.6977
Iteration: 6
Objective value: -9.6416
Iteration: 7
Objective value: -9.6416
Iteration: 8
Objective value: -9.6415
Iteration: 9
Objective value: -9.6414
Iteration: 10
Objective value: -9.6414

Final constraint violations: 1.0e-06 *
    0.6332    0.6236   -0.0779   -0.8792    0.2906

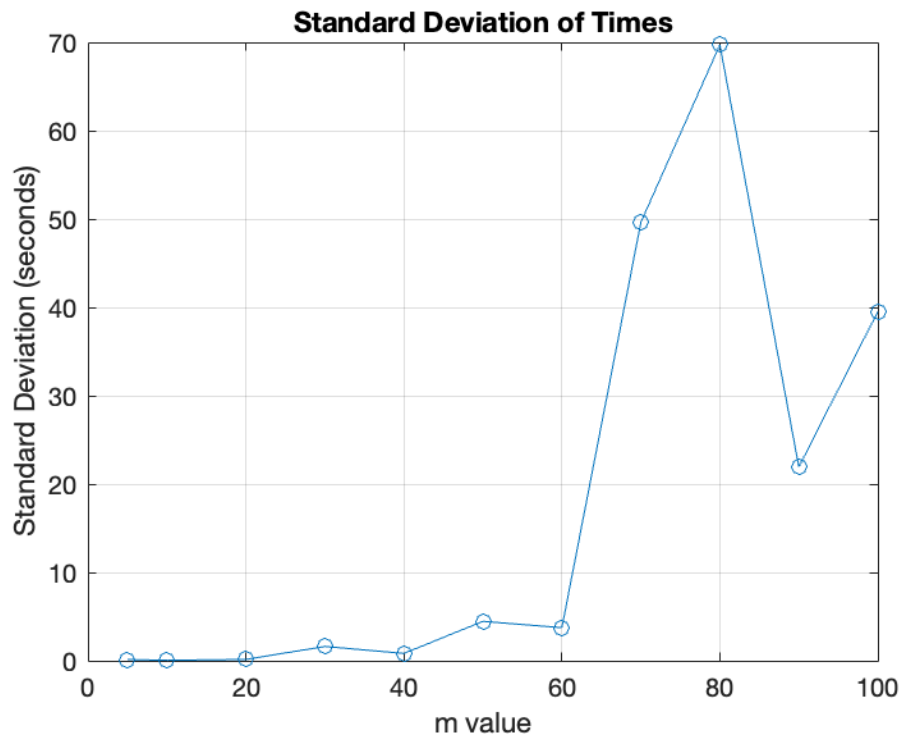
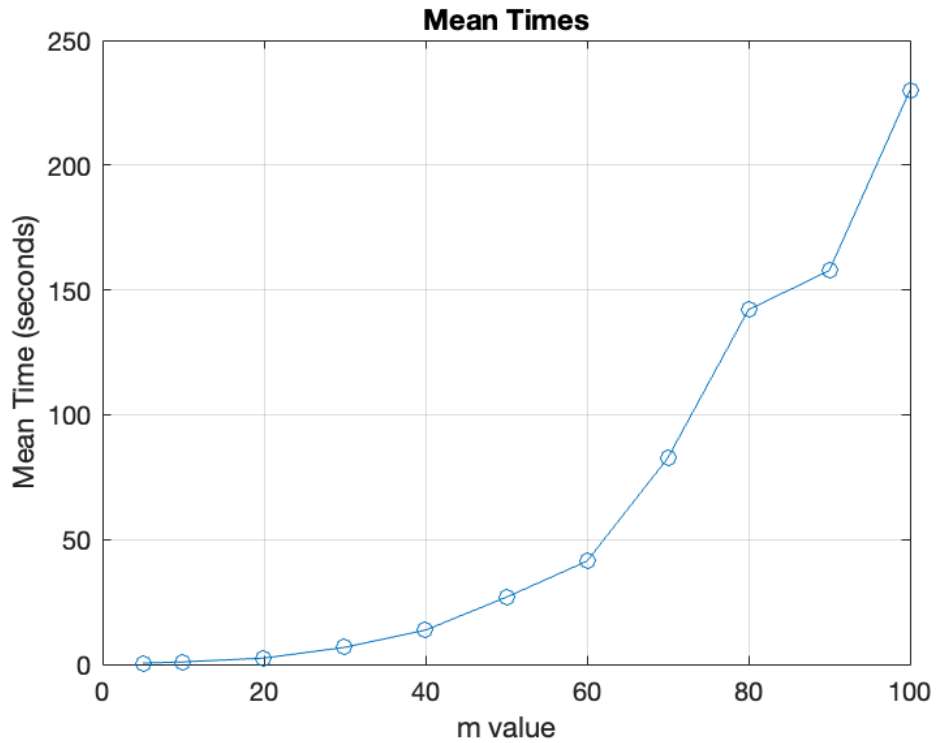
x_star =
   -0.5336
    0.3520
   -1.0000
    0.4515
   -0.0306
    0.2182
    0.1852
   -0.0921
   -0.3095
   -0.1253
>>
```

Problem 2.

I use the script ***“final_p2.m”*** to generate 10 random instances of the nonlinear program for varying values of m , and to record the mean time and standard deviation of the problem.

For this problem I used $m = 5, 10, 20, \dots, 100$, $\text{maxIter} = 100$.
And I set $\gamma = 0$ as we are looking at the convex quadratic case.

After running the script for m up to 100, I get the following plots:

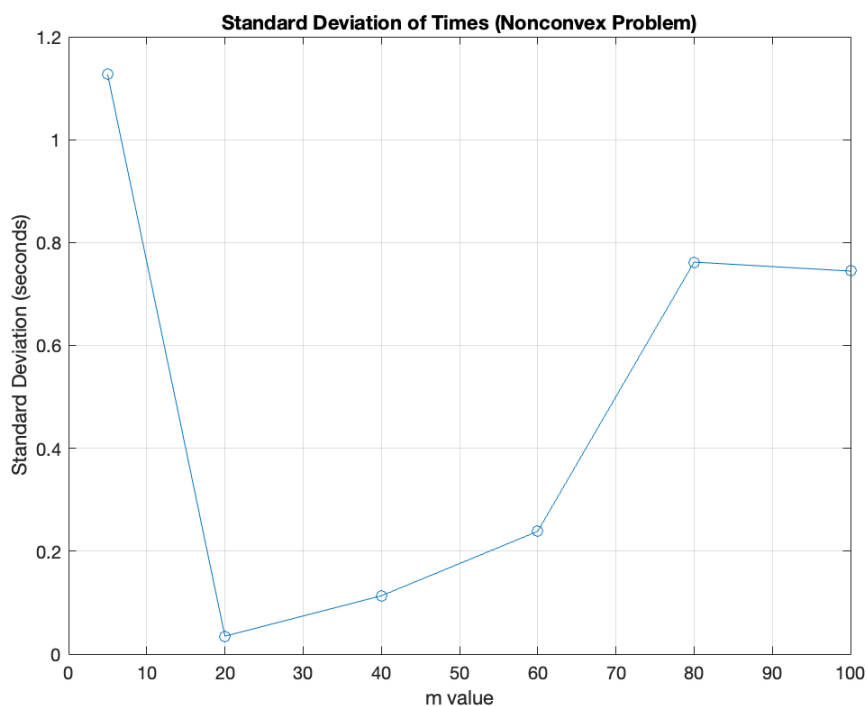
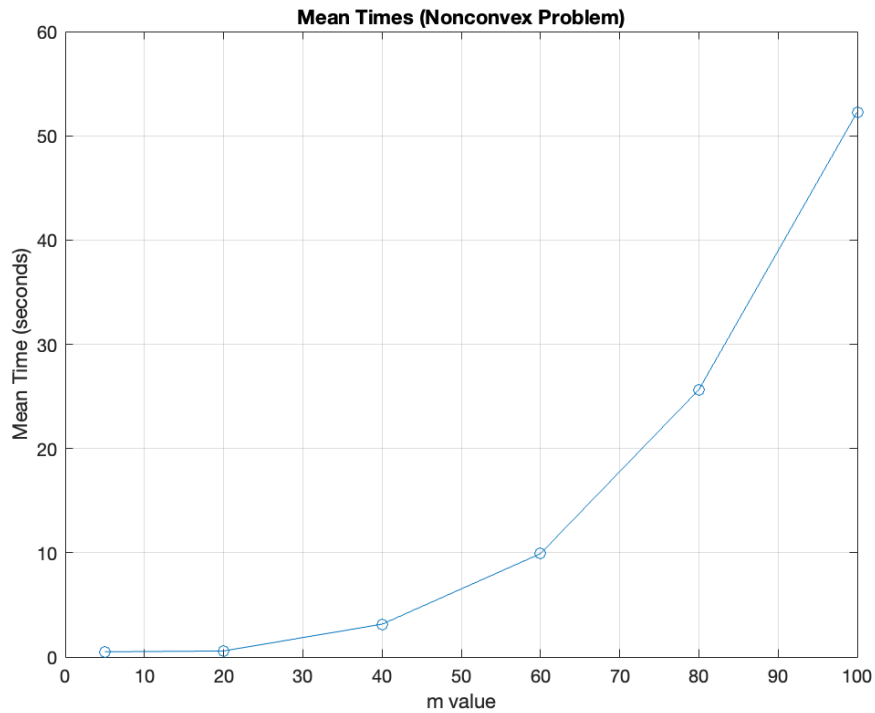


Problem 3.

I use the script ***“final_p3.m”*** to generate 10 random instances of the nonlinear program for varying values of m , and to record the mean time and standard deviation of the problem.

For this problem I used $m = 5, 20, 40, 60, 80, 100$, and $\text{maxIter} = 25$.
And I set $\gamma = 10$ as we are looking at the non-convex case.

After running the script for m up to 100, I get the following plots:



Problem 4.

A point x^* satisfies the second order sufficient condition if the projection of the Hessian of the Lagrangian onto the subspace tangent to the active constraints is positive definite.

In our specific problem we have equality constraints which are all active at the solution, and bound constraints which we can check whether they are active or not at the solution.

So given our solution x^* and λ^* , we check whether the projected Hessian onto the tangent space of the active constraints is positive definite or not to verify whether it satisfies the SOSC.

I implemented the function ***“checkSOSC(x, lambda, mu, c, Q0, D, a, gamma)”*** to perform this check for a given x and λ , and return a flag of whether or not this condition is satisfied.

Within the same function file, I also have the helper functions ***“hessLagrangian”*** to compute the Hessian of the Lagrangian (not the augmented one), and ***“identifyActiveConstraints”*** to determine the active constraints at the solution.

In script ***“final_p4.m”***, I generate 10 random instances of the program each for varying values of m from 5 to 50, obtain the solution, and check whether or not the SOSC is satisfied for each of them.

As we are interested in the non-convex case, I set $\gamma = 10$.

The script stores the success flags for each of these problems, and finally outputs a frequency result:

After running the script I get the following output:

```
>>
```

```
The SOSC is satisfied in 0.00% of the instances.
```

```
>>
```

However it is interesting to note that in the convex case ($\gamma = 0$), we do obtain that the SOSC is satisfied with a frequency of $> 0\%$.

One reason for this discrepancy may be due to the fact that the non-convex optimization is a harder problem to solve with many local minima / saddle points so our basic optimization algorithm may not converge to a minimum.

Further due to the added complexity, it may require tighter tolerances and many further iterations to converge to a point that satisfies the SOSC. A finer tuning of tolerances in the BCLMethod function may be needed to achieve convergence.

In addition a starting λ_0 of zeros for the multipliers may not be optimal for the non-convex problem.

Finally, in all the subproblems and functions, numerical issues with very small numbers can compound and lead to stability issues leading us to a non-optimal solution.

Code Submission.

The functions, scripts, and plots mentioned above are all saved in the folder *FINAL*.

There is also the function ***“final.m”*** with the calling sequence *“[x,obj]=final(m,n,s,p)”*.

The function generates a random linear programming problem of size m, n using the function *“RandomNonLinearProgram”* from above, and solves it using the *“BCLMethod”* algorithm.

The function has inputs:

```
% m,n - The dimensions of the problem
% s - The seed for the random number generator
% p - The maximum number of iterations for the simplex method
```

And outputs:

```
% x - The solution vector
% obj - The optimal objective function value
```

Note: gamma was not specified as a parameter, so by default I set it to 0, however this can be changed by adjusting the initialization at the top of the function *“final.m”*.

It should also be noted that if the maximum number of iterations has been reached before the bound constrained lagrangian method has converged to the specified tolerances in the function, the final value of x and the objective at the last step will be returned instead.

For example running the following command gives the output as below:

```
>> [x, obj] = final(5, 10, 6, 10)
```

```
Iteration: 1
```

```
Objective value: 0.0000
```

```
Iteration: 2
```

```
Objective value: -7.9117
```

```
Iteration: 3
```

```
Objective value: -11.1886
```

```
Iteration: 4
```

```
Objective value: -9.6977
```

```
Iteration: 5
```

```
Objective value: -9.6977
```

```
Iteration: 6
```

```
Objective value: -9.6416
```

```
Iteration: 7
```

```
Objective value: -9.6416
```

```
Iteration: 8
```

```
Objective value: -9.6415
```

```
Iteration: 9
```

```
Objective value: -9.6414
```

```
Iteration: 10
```

```
Objective value: -9.6414
```

```
Final constraint violations: 1.0e-06 *
```

```
0.6332 0.6236 -0.0779 -0.8792 0.2906
```

```
x =
```

```
-0.5336
```

```
0.3520
```

```
-1.0000
```

```
0.4515
```

```
-0.0306
```

```
0.2182
```

```
0.1852
```

```
-0.0921
```

```
-0.3095
```

```
-0.1253
```

```
obj =
```

```
-9.6414
```

```
>>
```